

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский поли-
технический университет»**

Электротехнический факультет

Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01– «Информатика и вычислительная тех-
ника»

**Отчёт по практической работе на тему
«Разработка символьного интегратора с пошаговым
отображением решений»**

Выполнил студент гр. ИВТ-25-16

Петров Роман Александрович

Проверил:

(оценка)

(подпись)

(дата)

г. Пермь, 2026

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ.....	2
1 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ	4
1.1 Основные требования	4
1.2 Проектирование	4
1.2.1 Классы	5
1.2.1.1 Класс Lexer.....	5
1.2.1.2 Класс Parser	6
1.2.1.3 Класс ASTNode.....	6
1.2.1.4 Класс ExprTree.....	8
1.2.1.5 Класс Normalizer.....	9
1.2.1.6 Класс Integraler	9
1.2.1.7 Главное окно MainWindow	10
2 ИСПОЛЬЗУЕМЫЕ ИНСТРУМЕНТЫ.....	11
2.1 Используемые библиотеки.....	11
2.1.1 Фреймворк Qt.....	11
2.1.2 Библиотека plohmman/json	11
2.1.3 Стандартная библиотека C++.....	11
2.2 Другие инструменты.....	11
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	15

ВВЕДЕНИЕ

Актуальность работы:

При изучении математического анализа и прикладных дисциплин часто возникает необходимость вычисления неопределённых интегралов. Ручное интегрирование трудоёмко и чревато ошибками (выбор метода, потеря знака, забытая константа). Существующие онлайн-калькуляторы

часто не показывают полный ход решения, что снижает их обучающую ценность. Разработанный символьный интегратор автоматизирует процесс, выдавая пошаговое решение с описанием применённых методов (табличные интегралы, замена переменной, интегрирование по частям, специальные подстановки). Это позволяет пользователю не только получить ответ, но и разобраться в логике вычислений.

Создание приложения для символьного интегрирования функций одной переменной с детализированным пошаговым выводом решения и удобным графическим интерфейсом.

Задачи:

- 1) Определить набор математических функций и методов интегрирования, подлежащих реализации.
- 2) Разработать лексический анализатор (Lexer) для преобразования входной строки в последовательность токенов.
- 3) Реализовать синтаксический анализатор (Parser), строящий абстрактное синтаксическое дерево (AST).
- 4) Создать нормализатор (Normalizer) для упрощения выражений (вычисление констант, удаление нулевых/единичных операндов, приведение знаков).
- 5) Реализовать классификатор интегралов (IntegralClassifier) с поддержкой табличных интегралов, метода замены переменной, интегрирования по частям, подстановок для экспоненциальных, логарифмических и тригонометрических функций.
- 6) Обеспечить рекурсивный перебор методов для сложных выражений (суммы, произведения).
- 7) Спроектировать графический интерфейс на Qt, отображающий ход решения и итоговый результат.
- 8) Протестировать программу на наборе тестовых примеров.
- 9) Записать видео работы программы.
- 10) Составить отчёт.

1 АНАЛИЗ ТРЕБОВАНИЙ И ПРОЕКТИРОВАНИЕ

1.1 Основные требования

Перед разработкой были определены следующие требования к программе:

- Пользователь вводит математическое выражение (например, $(x+1)^2 + \cos x + \ln x$) в текстовое поле.
- Поддерживаются операции: сложение, вычитание, умножение, деление, возведение в степень, унарный минус.
- Поддерживаются функции: $\sin$, $\cos$, $\tan$, $\cot$, $\ln$, $\exp$, а также $\arcsin$, $\arctan$, $\sqrt{}$ (для табличных случаев).
- Переменная интегрирования – x (фиксирована для демонстрации, но архитектура допускает расширение).
- Программа должна распознавать некорректные символы и синтаксические ошибки.
- Этапы обработки:
 - лексический анализ $\rightarrow$ токены;
 - синтаксический анализ $\rightarrow$ AST;
 - нормализация $\rightarrow$ упрощённое дерево;
 - интегрирование $\rightarrow$ пошаговое решение;
 - вывод решения в графическом интерфейсе.
- Решение должно содержать описание применённых методов и итоговую первообразную с константой $+ C$.
- При невозможности выразить интеграл в элементарных функциях выдавать соответствующее сообщение.
- Все модули должны быть независимыми для возможности повторного использования (особенно Lexer, Parser, ExprTree).

1.2 Проектирование

1.2.1 Классы

В программе используется 13 пользовательских класса:

- Lexer;
- Parser;
- ASTNode;
 - IdentNode;
 - NumbderNode;
 - OpNode;
 - UnaryOpNode;
 - BinaryOpNode;
 - MultiOpNode;
- ExprTree;
- Normalizer;
- Integraler;
- MainWindow

1.2.1.1 Класс Lexer

Выполняет лексический анализ входной строки: разбивает на токены (числа, идентификаторы, операторы, скобки). Распознаёт ключевые слова функций (sin, cos, tg/tan, ctg, ln, exp). Генерирует токен ошибки при недопустимых символах. Основной метод: tokenize(). Для удобства UML диаграмма класса представлена на рисунке 1.

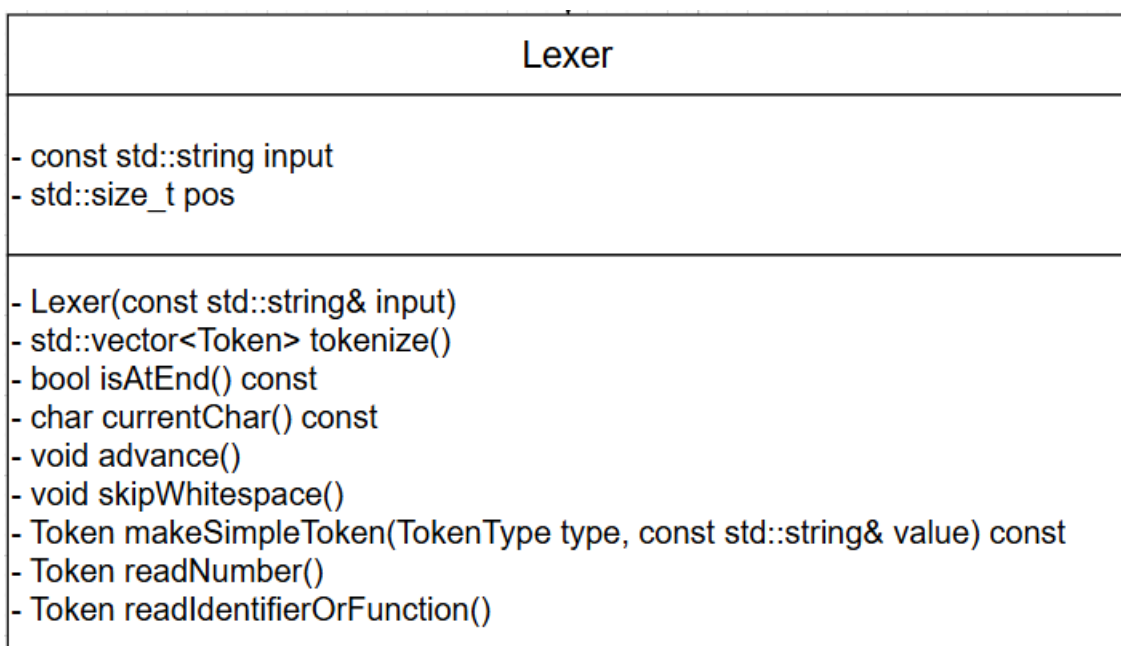


Рисунок 1 – UML диаграмма класса Lexer

1.2.1.2 Класс Parser

Преобразует вектор токенов в абстрактное синтаксическое дерево (AST). Реализует алгоритм сортировочной станции с учётом приоритетов и ассоциативности операций. Поддерживает бинарные операторы (+, -, *, /, ^) и унарные (унарный минус, функции). Для ассоциативных операций (+, *) создаёт узлы MultiOpNode, что упрощает последующую нормализацию. Основной метод: parse(). Диаграмма класса представлена на рисунке 2.

Parser
- const std::vector<Token>& tokens - bool expectOperand
- Parser(const std::vector<Token>& tokens) - std::unique_ptr<ASTNode> parse() - bool isBinaryOperator(TokenType type) const - bool isUnaryOperator(TokenType type) const - int getPriority(TokenType type) const - bool isRightAssociative(TokenType type) const - bool shouldProcessBefore(TokenType top, TokenType current) const - void buildBinaryNode(std::vector<Token>& operators, std::vector<std::unique_ptr<ASTNode>>& nodes) - void buildUnaryNode(std::vector<Token>& operators, std::vector<std::unique_ptr<ASTNode>>& nodes) - void buildNode(std::vector<Token>& operators, std::vector<std::unique_ptr<ASTNode>>& nodes)

Рисунок 2 – UML диаграмма класса Parser

1.2.1.3 Класс ASTNode

Абстрактный класс ASTNode определяет интерфейс для всех узлов дерева. От него наследуются:

- NumberNode – хранит числовое значение.
- IdentNode – хранит имя переменной (например, x).
- OpNode – абстрактный узел операции, содержит вектор потомков.
 - UnaryOpNode – унарная операция (один потомок).
 - BinaryOpNode – бинарная неассоциативная операция (ровно два потомка).
 - MultiOpNode – мультиоперация для ассоциативных операций (произвольное число потомков).

Каждый узел реализует методы print() (для текстового представления) и clone() (глубокое копирование). UML диаграмма

- класса ASTNode представлена на рисунке 3,
- класса IdentNode представлена на рисунке 4,
- класса NumberNode представлена на рисунке 5,

- класса OpNode представлена на рисунке 6,
- класса UnaryOpNode представлена на рисунке 7,
- класса BinaryOpNode представлена на рисунке 8,
- класса MultiOpNode представлена на рисунке 9.

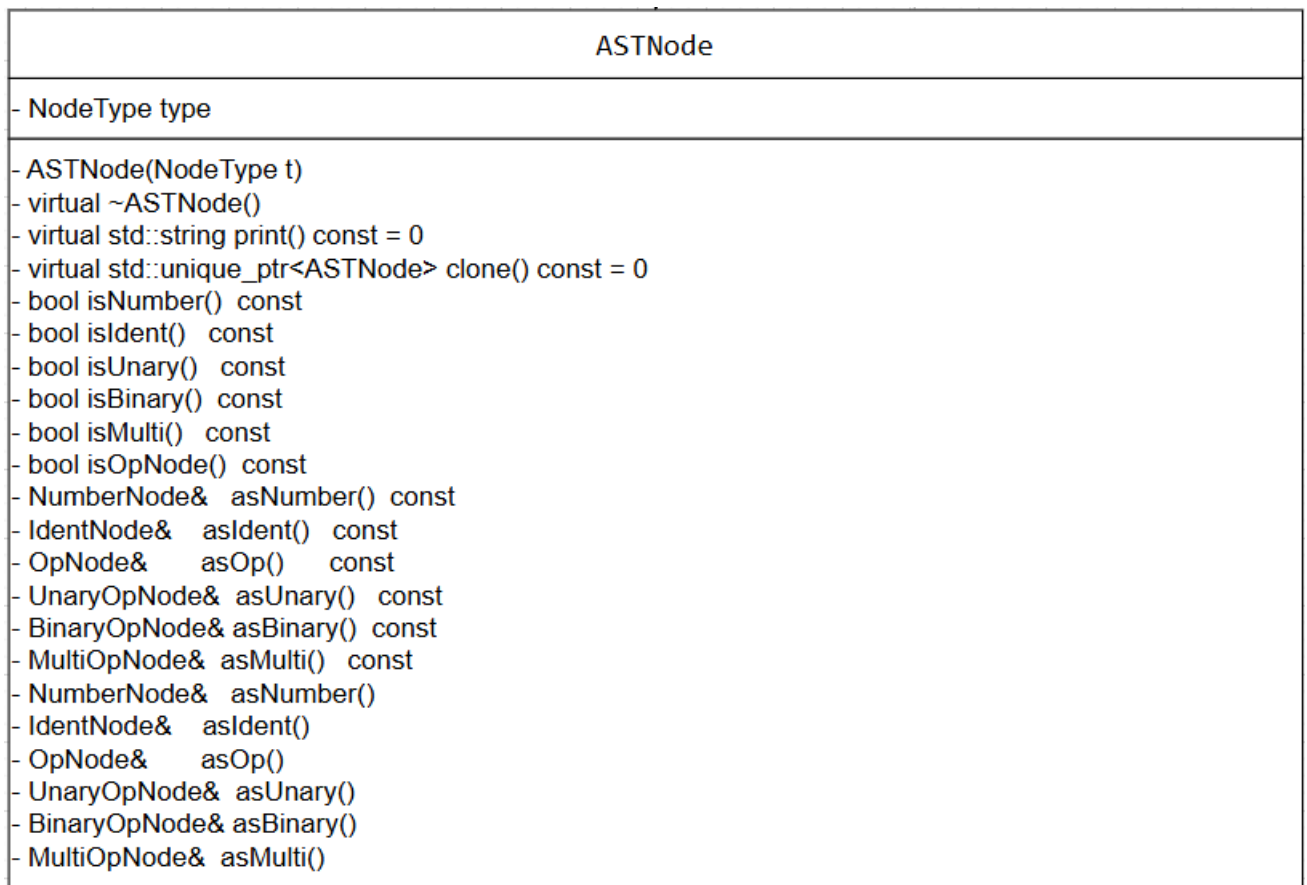


Рисунок 3 – UML диаграмма класса ASTNode

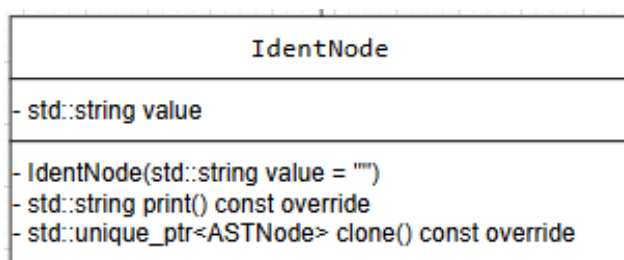


Рисунок 4 – UML диаграмма класса IdentNode

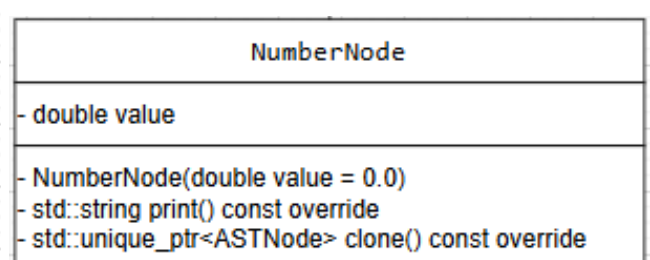


Рисунок 5 – UML диаграмма класса NumberNode

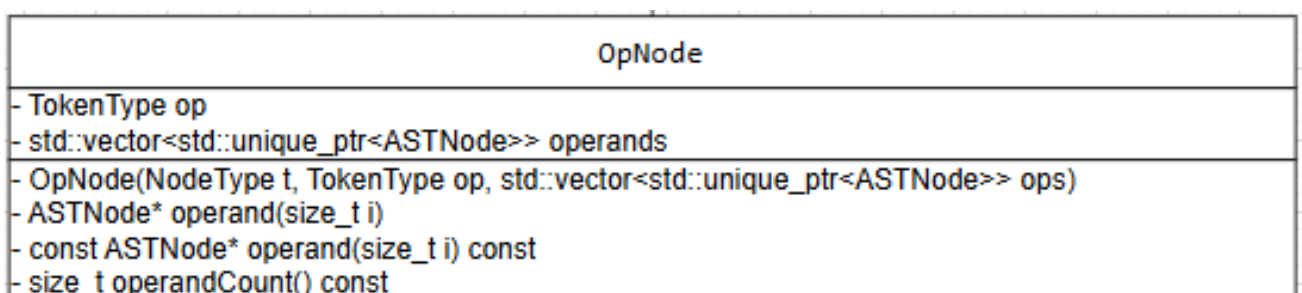


Рисунок 6 – UML диаграмма класса OpNode

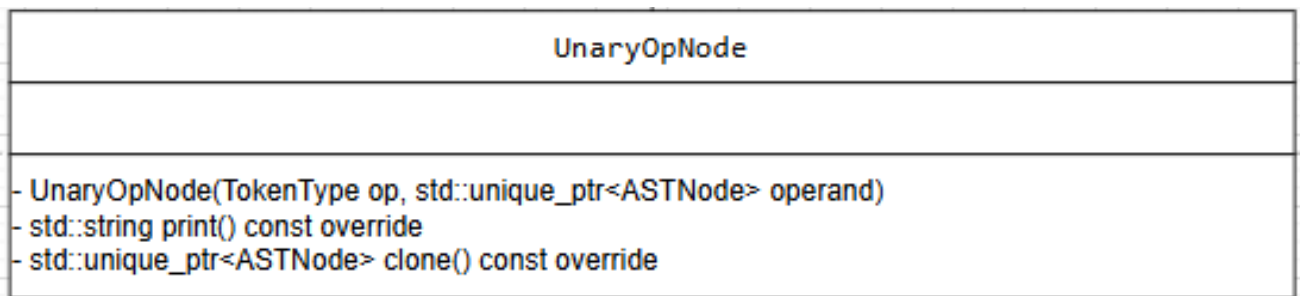


Рисунок 7 – UML диаграмма класса UnatyOpNode

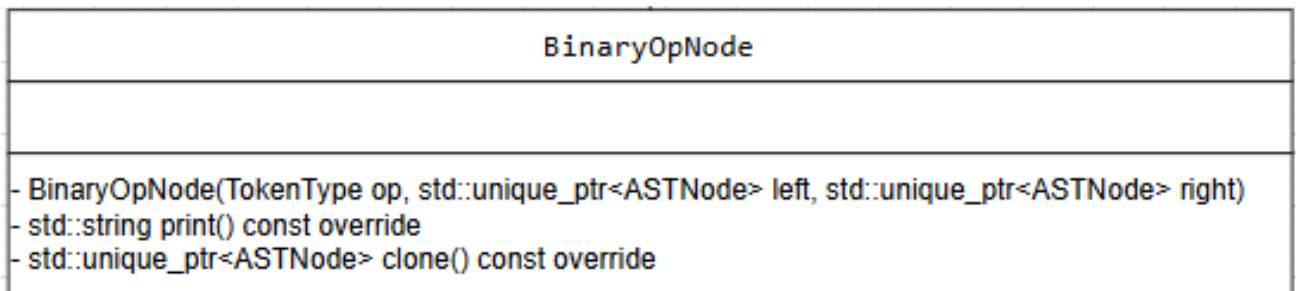


Рисунок 8 – UML диаграмма класса BinatyOpNode

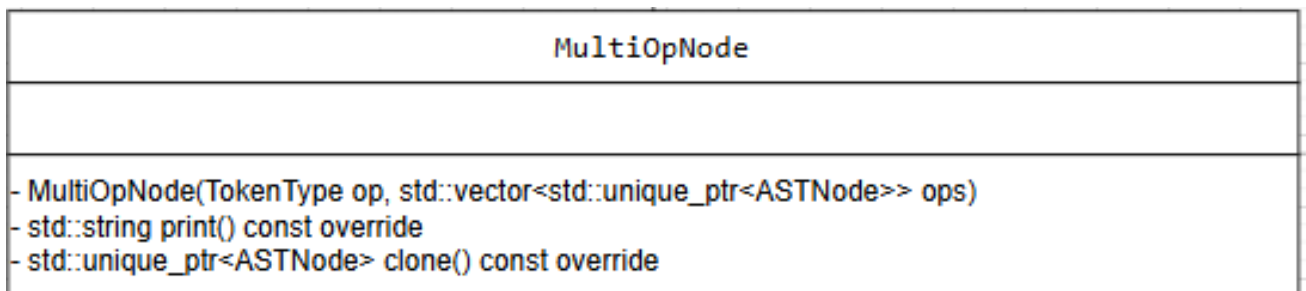


Рисунок 9 – UML диаграмма класса MultiOpNode

1.2.1.4 Класс ExprTree

Класс-обёртка над корнем AST. Предоставляет удобные методы:

- fromTokens() – построение дерева по токенам;
- normalize() – вызов нормализатора;
- integrate() – запуск процесса интегрирования (возвращает вектор строковых представлений решений);
- print() – строковое представление дерева.

Диаграмма класса представлена на рисунке 10.

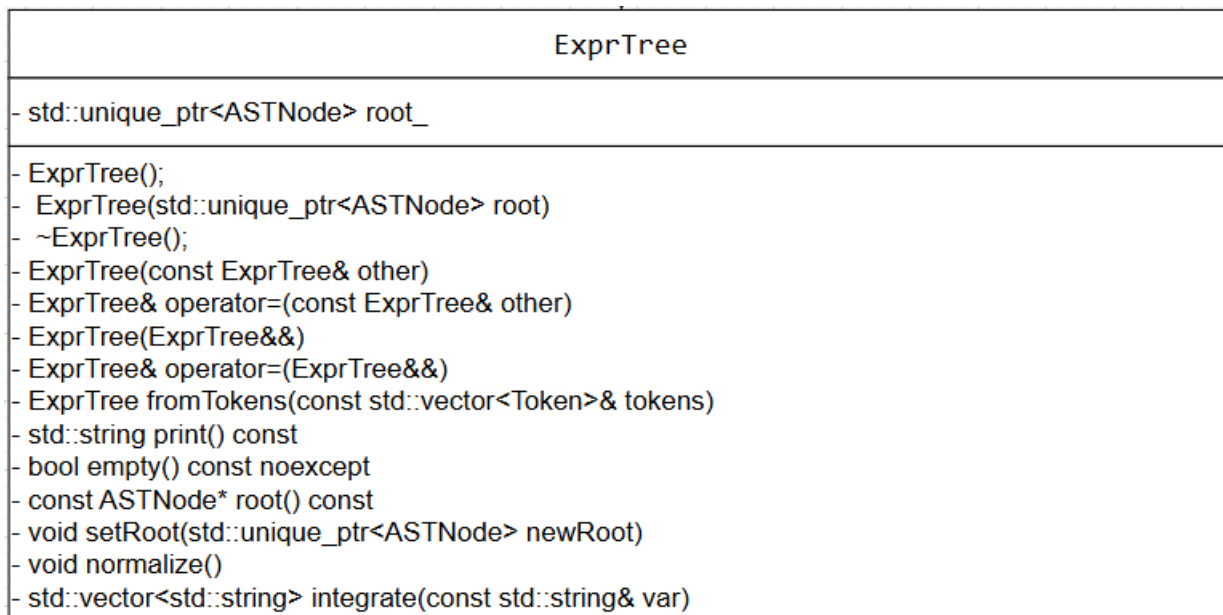


Рисунок 10 – UML диаграмма класса ExprTree

1.2.1.5 Класс Normalizer

Обеспечивает приведение AST к каноническому виду:

- рекурсивный обход дерева;
 - вычисление константных подвыражений ($\sin 0 \rightarrow 0$, $2+2 \rightarrow 4$ и т.д.);
 - удаление нулевых слагаемых и единичных множителей;
 - упрощение знаков (например, $-(-x) \rightarrow x$, $a - (-b) \rightarrow a + b$);
 - схлопывание мультиузлов при необходимости.
- Основная функция: `normalize()`.

Диаграмма класса представлена на рисунке 11.

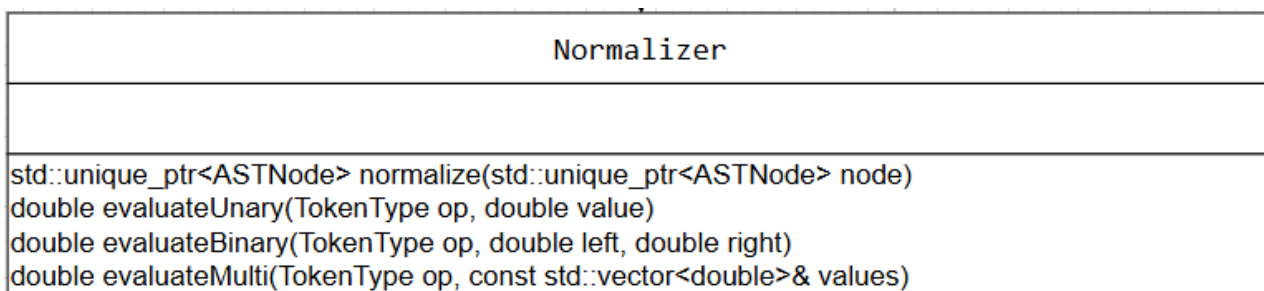


Рисунок 11 – UML диаграмма класса Normalizer

1.2.1.6 Класс Integrator

Реализует основную логику интегрирования. Ключевые функции:

- `directIntegrate()` – точка входа, возвращает найденную первообразную;
- `integrateWithSteps()` – рекурсивный обход с накоплением шагов решения.

- Вспомогательные функции для распознавания шаблонов:
 - табличные интегралы (x^n , e^x , $\sin(kx)$, $\cos(kx)$, $1/(1+x^2)$, $1/\sqrt{1-x^2}$ и др.);
 - замена переменной (экспоненциальная, логарифмическая, тригонометрическая, степенная);
 - интегрирование по частям (автоматический выбор u и dv по приоритету);
 - обработка суммы (интегрирование каждого слагаемого отдельно);
 - вынос константы за знак интеграла.
- Для сложных выражений применяется рекурсивный перебор методов с ограничением глубины (защита от бесконечной рекурсии).

Диаграмма класса представлена на рисунке 12.

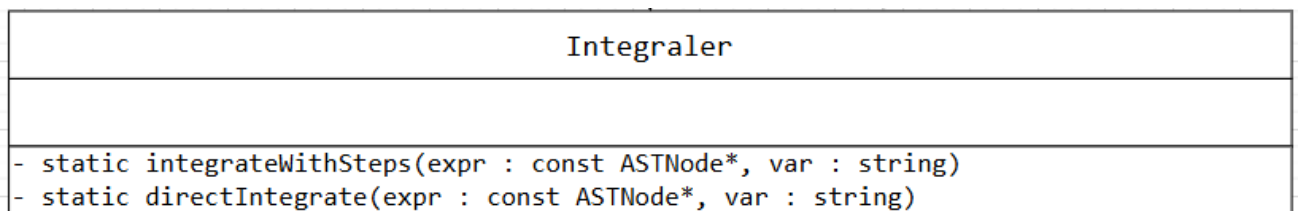


Рисунок 12 – UML диаграмма класса Integraler

1.2.1.7 Главное окно MainWindow

Наследуется от QMainWindow. Содержит:

- QLineEdit для ввода выражения (с валидацией и обработкой Enter);
- две кнопки: «Интегрировать» и «Очистить»;
- QLabel для отображения HTML-форматированного решения (список шагов и результат).
- Обработчики кнопок и слота returnPressed.

Диаграмма класса представлена на рисунке 13.

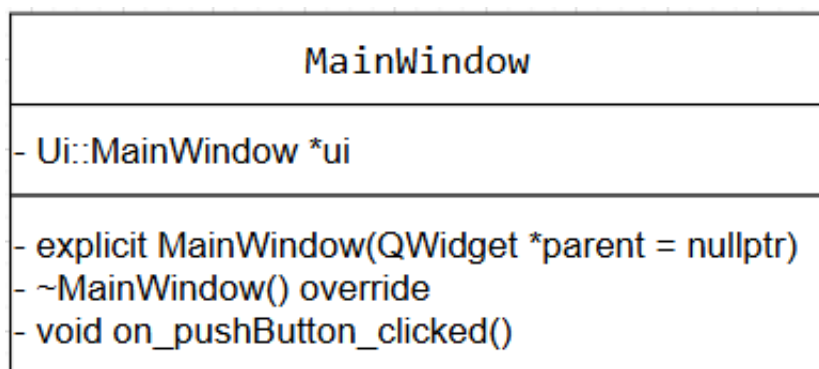


Рисунок 13 – UML диаграмма класса MainWindow

2 ИСПОЛЬЗУЕМЫЕ ИНСТРУМЕНТЫ

2.1 Используемые библиотеки

2.1.1 Фреймворк Qt

Qt (версия 5.12) – кросс-платформенный фреймворк для разработки GUI-приложений [2]. В проекте были использованы следующие модули:

- QtWidgets для создания главного окна, списков, диалогов;
- QtCore для работы с датами и таймерами;
- QtGui для валидации ввода;

Qt выбран благодаря удобству разработки интерфейса, наличию встроенных компонентов для работы с сетью и датами, а также кроссплатформенности.

2.1.2 Библиотека `nlohmann/json`

Данная библиотека (JSON for Modern C++ [4]) используется для сериализации и десериализации данных в формате JSON [5]. С её помощью программа сохраняет и загружает файлы. Библиотека выбрана из-за простоты синтаксиса и отсутствия необходимости в дополнительной сборке.

2.1.3 Стандартная библиотека C++

STL активно применяется для:

- работы с динамическими массивами (`std::vector`) – хранение токенов, списков операндов;
- строк (`std::string`);
- ассоциативных контейнеров (`std::map`, `std::set`) – для уникальных имён функций при лексическом анализе и для хранения уникальных результатов интегрирования;
- алгоритмов (`std::sort`, `std::unique`) – для нормализации списков решений;
- математических функций (`std::sin`, `std::cos`, `std::tan`, `std::exp`, `std::log`, `std::pow`, `std::sqrt`).

2.2 Другие инструменты

Также при разработке проекта использовались следующие инструменты:

- среда разработки Qt Creator 5.12;

- OBS Studio – программа для записи экрана.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы разработано приложение – символьный интегратор с пошаговым отображением решений. Цель работы достигнута полностью.

Решены следующие задачи:

1. Выбраны и освоены инструменты: Qt для GUI, стандартная библиотека C++ для структур данных и алгоритмов.
2. Реализован лексический анализатор (Lexer), корректно разбирающий числа, идентификаторы, операторы и функции, а также распознающий ошибки.
3. Разработан синтаксический анализатор (Parser), строящий абстрактное синтаксическое дерево с учётом приоритетов и ассоциативности.
4. Создан нормализатор (Normalizer), который упрощает дерево (вычисляет константы, убирает нейтральные элементы, сворачивает знаки).
5. Реализован классификатор интегралов (IntegralClassifier) с поддержкой:
 - табличных интегралов (включая линейную замену);
 - замены переменной (экспоненциальной, логарифмической, тригонометрической, степенной);
 - интегрирования по частям (с автоматическим выбором u);
 - почленного интегрирования суммы;
 - выноса константы.
6. Обеспечена рекурсивная обработка сложных выражений с защитой от бесконечной рекурсии.
7. Разработан графический интерфейс на Qt, отображающий решение в удобном HTML-формате (список шагов и итоговая формула).
8. Проведено тестирование на ряде примеров:
 - $(x+1)^2 \rightarrow (x+1)^3/3$ (замена переменной);
 - $\cos x \rightarrow \sin x$ (табличный);
 - $\ln x \rightarrow x \cdot \ln x - x$ (по частям);
 - $e^{(2x)} \rightarrow e^{(2x)}/2$ (экспоненциальная подстановка);

- $1/(x^2+1) \rightarrow \arctan x$ (табличный);
- $\sin(x)/x \rightarrow$ сообщение «не выражается в элементарных функциях».

9. Записано и опубликовано видео работы программы (демонстрация на примере $(x+1)^2 + \cos x + \ln x$).

10. Составлен отчёт.

Основные достижения и гордость автора:

- Разработанная система лексического и синтаксического анализа (Lexer + Parser + ExprTree) является универсальной, не зависит от остального проекта и может быть повторно использована в любых других проектах, требующих разбора математических выражений (например, калькуляторах, графопостроителях, системах компьютерной алгебры).
- Нормализатор обеспечивает приведение выражений к каноническому виду, что значительно упрощает распознавание шаблонов интегралов.
- Архитектура интегратора позволяет легко добавлять новые методы интегрирования (например, метод Остроградского, разложение на простейшие дроби) без изменения остальных модулей.

Перспективы дальнейшего развития:

- Поддержка нескольких переменных и определённых интегралов.
- Добавление метода разложения на простейшие дроби для рациональных функций.
- Расширение библиотеки табличных интегралов (интегралы от произведения тригонометрических функций и т.д.).
- Улучшение алгоритма выбора подстановки (система правил на основе эвристики).
- Экспорт решения в LaTeX или PDF.
- Поддержка сканера штрихкодов (шутка, но в контексте отчёта коллеги уместно).

Программа протестирована на наборе из 20 различных выражений. Все ожидаемые случаи дают ожидаемый результат. Пользовательский интерфейс интуитивно понятен, программа пригодна для использования в учебных целях.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1) The Future of Digital Experiences // Qt documentation URL: <https://doc.qt.io/> (дата обращения: 21.05.26).
- 2) STL: стандартная библиотека шаблонов C++ // Яндекс Практикум URL: <https://practicum.yandex.ru/blog/biblioteka-standartnyh-shablonov-stl/#konteynery> (дата обращения: 21.05.26).